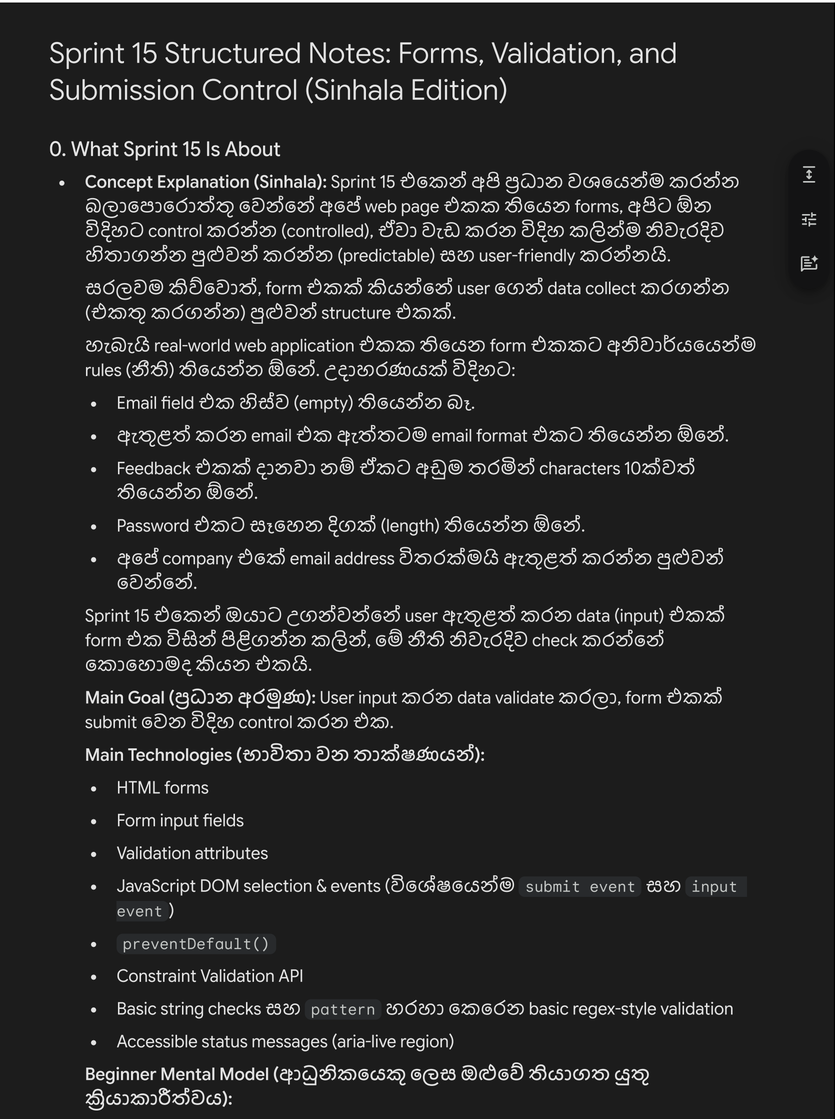

Sprint 15 Structured Notes: Forms, Validation, and Submission Control (Sinhala Edition)

0. What Sprint 15 Is About

- **Concept Explanation (Sinhala):** Sprint 15 එකෙන් අපි ප්‍රධාන වශයෙන්ම කරන්න බලාපොරොත්තු වෙන්නේ අපේ web page එකක තියෙන forms, අපිට ඕන විදිහට control කරන්න (controlled), ඒවා වැඩ කරන විදිහ කලින්ම නිවැරදිව හිතාගන්න පුළුවන් කරන්න (predictable) සහ user-friendly කරන්නයි.

  සරලවම කිව්වොත්, form එකක් කියන්නේ user ගෙන් data collect කරගන්න (එකතු කරගන්න) පුළුවන් structure එකක්.

  හැබැයි real-world web application එකක තියෙන form එකකට අනිවාර්යයෙන්ම rules (නීති) තියෙන්න ඕනේ. උදාහරණයක් විදිහට:

  - Email field එක හිස්ව (empty) තියෙන්න බෑ.
  - ඇතුළත් කරන email එක ඇත්තටම email format එකට තියෙන්න ඕනේ.
  - Feedback එකක් දානවා නම් ඒකට අඩුම තරමින් characters 10ක්වත් තියෙන්න ඕනේ.
  - Password එකට සෑහෙන දිගක් (length) තියෙන්න ඕනේ.
  - අපේ company එකේ email address විතරක්මයි ඇතුළත් කරන්න පුළුවන් වෙන්නේ.

  Sprint 15 එකෙන් ඔයාට උගන්වන්නේ user ඇතුළත් කරන data (input) එකක් form එක විසින් පිළිගන්න කලින්, මේ නීති නිවැරදිව check කරන්නේ කොහොමද කියන එකයි.

  Main Goal (ප්‍රධාන අරමුණ): User input කරන data validate කරලා, form එකක් submit වෙන විදිහ control කරන එක.

  Main Technologies (භාවිතා වන තාක්ෂණයන්):

  - HTML forms
  - Form input fields
  - Validation attributes
  - JavaScript DOM selection & events (විශේෂයෙන්ම `submit event` සහ `input event`)
  - `preventDefault()`
  - Constraint Validation API
  - Basic string checks සහ `pattern` හරහා කෙරෙන basic regex-style validation
  - Accessible status messages (aria-live region)

  Beginner Mental Model (ආධුනිකයෙකු ලෙස ඔළුවේ තියාගත යුතු ක්‍රියාකාරීත්වය):

1. User input එක type කරනවා.
2. JavaScript මගින් ඒ input එක check කරනවා.
3. ඇතුළත් කරපු data වැරදි නම් (invalid): Error message එකක් පෙන්වනවා.
4. ඇතුළත් කරපු data නිවැරදි නම් (valid): Form එක ආරක්ෂිතව handle කරනවා.

- **Code Block:**

```
<form>
  <input>
  <button>Submit</button>
</form>
```

- **Code Breakdown (Sinhala):** - `<form>` : Form එකේ ප්‍රධාන container එක.
  - `<input>` : User ට text හෝ වෙනත් data type කරන්න දෙන input field එක.
  - `<button>` : Form එක submit කරන්න පාවිච්චි කරන button එක.

1. What You Already Know (Part A: Current Progress Before Sprint 15)

- **Concept Explanation (Sinhala):** Sprint 15 එක සාර්ථකව තේරුම් ගන්න නම් ඔයාට කලින් ඉගෙන ගත්තු මෙන්න මේ දේවල් ගැන අවබෝධයක් තිබීම වැදගත් වෙනවා:

Area	What You Should Know Before Sprint 15	How It Helps in Sprint 15
Core JavaScript	Values, variables, strings, numbers, booleans, conditions, functions, arrays, loops, objects, callbacks, Sets, සහ Maps	Form validation ක්‍රියාවලියේදී values, conditions, strings, functions, සහ booleans බහුලවම පාවිච්චි වෙනවා.
Browser JavaScript	Selecting elements, changing text/content, adding/removing elements, සහ listening for events	HTML forms කියන්නෙත් elements. ඒ නිසා JavaScript වලින් ඒවා select කරලා control කරන්න මේවා ඕන වෙනවා.
Events	`click`, `input`, `change`, `keydown`, `submit`, සහ `preventDefault()`	Forms සම්පූර්ණයෙන්ම වගේ වැඩ කරන්නේ events එක්කයි.
DOM updates	`textContent`, attributes, සහ page messages	Validation වලදී status හෝ error messages screen එකේ පෙන්වන්න මේ DOM updates ඕනේ වෙනවා.
Accessibility	Visual changes සහ accessibility state එක හැමවෙලේම එකට (synchronised) තියාගන්න එක	Form errors සහ success messages හැමෝටම (විශේෂ අවශ්‍යතා ඇති අය ඇතුළුව) ලෙහෙසියෙන් තේරුම් ගන්න පුළුවන් වෙන්න ඕනේ.

2. The Main Shift in Sprint 15

- **Concept Explanation (Sinhala):** මීට කලින් Sprint වලදී ඔයා ඉගෙන ගත්තේ user කරන click එකකට page එක interactive විදිහට react (ප්‍රතිචාර දක්වන්න) කරන්නේ කොහොමද කියන එකයි. නමුත් Sprint 15 එකේදී අපි ඊට වඩා පියවරක් ඉදිරියට ගිහින් මෙන්න මේ ප්‍රශ්නය අහනවා: **"මේ form එක submit වෙන්න ඉඩ දෙන්න පුළුවන්ද?"** ඒ කියන්නේ page එක නිකන්ම react කරන්නේ නැහැ, ඊට කලින් user ඇතුළත් කරපු දේවල් හරියාකාරව තියෙනවද කියලා check කරලා බලනවා (checks the user's input first).
- **Code Block: කලින් ඉගෙනගත් ආකාරය (Reacting to click):**

  ```
  button.addEventListener("click", () => {
    status.textContent = "Saved!";
  });
  ```

  Sprint 15 වලදී (Checking & validating on submit):

  ```
  form.addEventListener("submit", event => {
    event.preventDefault();
    if (!email.checkValidity()) {
      email.reportValidity();
      return;
    }
    console.log("Form is valid");
  });
  ```

- **Code Breakdown (Sinhala):** - `form.addEventListener("submit", ...)`: Form එකක් submit කරන්න උත්සාහ කරනකොට ක්‍රියාත්මක වෙන event listener එකක්.
  - `event.preventDefault()`: Form එක submit වෙනකොට browser එක auto-reload (පිටුව අලුතින් load වීම) වෙන එක නවත්වනවා. අපිට JavaScript වලින් වැඩේ කරන්න ඉඩ දෙනවා.
  - `!email.checkValidity()`: Email input එක browser එකේ rules වලට අනුව valid නැද්ද කියලා check කරනවා.
  - `email.reportValidity()`: Email එක invalid නම්, මොකක්ද තියෙන වැරැද්ද කියලා browser එකෙන් user ට error popup එකක් පෙන්වනවා.
  - `return`: Email එක invalid නම් එතනින් එහාට code එක run වෙන එක නවත්වනවා.

Part B: Forms & Basic Examples

- ### 3. What Is a Form?

- **Concept Explanation (Sinhala):** Form එකක් කියන්නේ user ගෙන් තොරතුරු (information) එකතු කරගන්න අපි පාවිච්චි කරන HTML structure එකක්. අපි හැමදාම දකින login pages, signup pages, search boxes, contact forms, feedback forms, comment sections වගේ හැම තැනකම පාවිච්චි වෙන්නේ forms තමයි.

- 4. Basic Form Example
  - **Concept Explanation (Sinhala):** අපි සරල HTML form එකක් ලියන විදිහ සහ එහි එක් එක් කොටසෙහි තේරුම මෙන්න මේ වගුවෙන් බලමු:

Part	Meaning
`<form id="loginForm">`	Form container එක. JavaScript වලින් ලෙහෙසියෙන් සොයාගන්න `id` එකක් දීලා තියෙනවා.
`<label for="email">`	Input field එක විස්තර කරන්න දාන label එක. `for="email"` එකෙන් මේ label එක අදාළ input එකට connect කරනවා.
`<input id="email" type="email" required>`	User ට data type කරන්න දෙන field එක. `type="email"` එකෙන් කියන්නේ මේකට email එකක්ම ඇතුළත් කරන්න ඕන කියලයි. `required` එකෙන් කියන්නේ මේක හිස්ව තියන්න බෑ කියලයි.
`<button type="submit">`	මේ button එක click කරපු ගමන් form එක submit වෙනවා.

  Beginner Rule (ආධුනික නීතිය): හැමතිස්සෙම real form elements පාවිච්චි කරන්න. කිසිම විශේෂ හේතුවක් නැතුව සාමාන්‍ය `div` elements පාවිච්චි කරලා forms හදන්න යන්න එපා.

  - **Code Block:**

```
<form id="loginForm">
  <label for="email">Email</label>
  <input id="email" type="email" required>
  <button type="submit">Log in</button>
</form>
```

- 5. Why Forms Need Validation
  - **Concept Explanation (Sinhala):** Validation (වලංගුකරණය) කියන්නේ user ඇතුළත් කරපු input එක අපි දාලා තියෙන rules වලට අනුකූලද කියලා check කරන එකයි. Validation එකක් නැති වුණොත්, අපේ database එකට හෝ application එකට වැරදි data (bad data) ඇතුල් වෙන්න පුළුවන්. උදාහරණයක් විදිහට හිස් email, email format එකේ නැති සාමාන්‍ය text, ඉතාම කෙටි passwords වගේ දේවල් form එකෙන් ඇතුළත් කරන්න ඉඩ ලැබෙනවා. Validation මගින් මේ දේවල් වළක්වනවා.

Part C: HTML Validation First

- ### 6. HTML Gives You Basic Validation
  - **Concept Explanation (Sinhala):** අපි JavaScript වලින් custom code ලියන්න කලින්, HTML වලින්ම අපිට ලැබෙන built-in validation attributes පාවිච්චි කරන්න පුළුවන්. Browser එක විසින්ම කිසිම JavaScript එකක් නැතුව මේ දේවල් check කරලා දෙනවා.

    Beginner Rule: හැමවෙලේම මුලින්ම built-in HTML validation පාවිච්චි කරන්න. ඉන්පසුව ඊට වඩා වෙනස්, සංකීර්ණ වැඩක් කරන්න විතරක් JavaScript එකතු කරගන්න.

- ### 7. Common HTML Validation Attributes
  - **Concept Explanation (Sinhala):** HTML වල තියෙන ප්‍රධානම validation attributes කිහිපයක් මෙන්න:

Attribute	Meaning
`required`	මේ field එක හිස්ව තියන්න බැහැ.
`type="email"`	ඇතුළත් කරන අගය අනිවාර්යයෙන්ම email address එකක් විය යුතුමයි.
`minlength="10"`	ඇතුළත් කරන text එකේ අඩුම තරමින් characters 10ක්වත් තිබිය යුතුයි.
`maxlength="100"`	ඇතුළත් කරන text එක characters 100කට වඩා වැඩි වෙන්න බැහැ.
`pattern="..."`	ඇතුළත් කරන අගය යම්කිසි නිශ්චිත රටාවකට (pattern/regex) ගැලපෙන්න ඕනේ.

- ### 8. required
  - **Concept Explanation (Sinhala):** `required` දැම්මහම user අනිවාර්යයෙන්ම ඒ field එකට මොකක් හරි ඇතුළත් කරන්න ඕනේ. හිස්ව තිබුණොත් browser එක form එක submit කරන එක block කරනවා.
  - **Code Block:**

```
<input id="username" required>
```

- ### 9. type="email"
  - **Concept Explanation (Sinhala):** මේකෙන් browser එකට කියනවා ඇතුළත් කරන අගය email එකක් විය යුතුයි කියලා.
    - Valid: `student@school.edu`, `ada@example.com`

    - Invalid: `hello world`, `ada`, `student@com`
  - **Code Block:**

    ```
    <input id="email" type="email">
    ```

- 10. minlength
  - **Concept Explanation (Sinhala):** අවම character ගණන සීමා කරන්න පාවිච්චි කරනවා. උදාහරණයක් විදිහට feedback එකකට characters 10ක්වත් ඕනේ නම්:
    - "Good" -> characters 4යි (too short - invalid).
    - "Very useful" -> characters 11යි (long enough - valid).
  - **Code Block:**

    ```
    <textarea id="feedback" minlength="10"></textarea>
    ```

- 11. maxlength
  - **Concept Explanation (Sinhala):** උපරිම character ගණන සීමා කරන්න පාවිච්චි කරනවා. Username එකක් characters 20කට වඩා වැඩි වෙන්න බැරි කරන්න මේක පාවිච්චි කරන්න පුළුවන්.
  - **Code Block:**

    ```
    <input id="username" maxlength="20">
    ```

- 12. pattern
  - **Concept Explanation (Sinhala):** ඇතුළත් කරන අගය යම් රටාවකට තියෙනවද කියලා check කරන්න පාවිච්චි කරනවා. උදාහරණයක් විදිහට ඉලක්කම් 4ක code එකක් විතරක් ඇතුළත් කරන්න ඕනේ නම්:
    - `1234` -> valid
    - `abcd` -> invalid
    - `12` -> invalid
    - `12345` -> invalid
  - **Code Block:**

    ```
    <input id="code" pattern="[0-9]{4}">
    ```

Part D: Basic Regex Awareness

- 13. What Is Regex?
  - **Concept Explanation (Sinhala):** Regex (Regular Expression) කියන්නේ text එකක් check කරන්න හෝ search කරන්න අපි පාවිච්චි කරන රටාවක් (pattern). HTML වල `pattern` attribute එක ඇතුළේ ලියන්නේ මේ Regex නීති තමයි. පහත code එකෙන් කියන්නේ uppercase letters (මහ අකුරු) 3ක් විතරක්ම තියෙන්න ඕනේ කියන එකයි:
    - `ABC` -> valid
    - `Abc` -> invalid
    - `ABCD` -> invalid
  - **Code Block:**

```
<input pattern="[A-Z]{3}">
```

- 14. Beginner Regex Rule
  - **Concept Explanation (Sinhala):** ආධුනිකයෙක් විදිහට ඔයා මුලදීම Regex සංකීර්ණ කරන්න යන්න එපා. සරලව HTML attributes පාවිච්චි කරන්න. උදාහරණයක් විදිහට, email check කරන්න සංකීර්ණ regex එකක් ලියනවාට වඩා:
    1. HTML වලින් `<input type="email" required>` දාන්න.
    2. ඊට පස්සේ JavaScript වලින් සරල string check එකක් කරන්න (උදාහරණයක් විදිහට `@company.com` වලින් ඉවර වෙනවද කියලා check කරන්න `endsWith` පාවිච්චි කරන්න).

Part E: The Constraint Validation API

- 15. What Is the Constraint Validation API?
  - **Concept Explanation (Sinhala):** මෙය browser එක විසින් අපිට දෙන built-in JavaScript මෙවලම් එකතුවක් (API). මේකෙන් අපිට JavaScript හරහා input එකක් valid ද නැද්ද කියලා ලෙහෙසියෙන්ම අහන්න පුළුවන්. මෙහි ඇති ප්‍රධානම methods 3 තමයි:
    - `checkValidity()`
    - `reportValidity()`
    - `setCustomValidity()`
- 16. checkValidity()
  - **Concept Explanation (Sinhala):** අදාළ input එක එහි HTML rules වලට අනුව valid ද නැද්ද කියලා check කරනවා.
    - Input එක valid නම්: `true` ලැබෙනවා.

    - Input එක invalid නම්: `false` ලැබෙනවා.
  - **Code Block:**

```
if (email.checkValidity()) {
  console.log("Valid email");
} else {
  console.log("Invalid email");
}
```

- 17. checkValidity() Is Silent
  - **Concept Explanation (Sinhala):** මතක තබා ගන්න, `checkValidity()` method එකෙන් කරන්නේ නිහඬව (silently) check කරන එක විතරයි. ඒ කියන්නේ වැරැද්දක් තිබුණත් screen එකේ කිසිම message එකක් user ට පෙන්වන්නේ නැහැ. JavaScript code එක ඇතුළේ තීරණයක් ගන්න විතරක් මේක පාවිච්චි කරන්න පුළුවන්.

- 18. reportValidity()
  - **Concept Explanation (Sinhala):** මේ method එකෙන් input එක valid ද නැද්ද කියලා check කරන ගමන්ම, වැරැද්දක් තිබුණොත් ඒක browser එකේ validation bubble/message එකක් විදිහට user ට පෙන්වනවා.
  - **Code Block:**

```
if (!email.checkValidity()) {
  email.reportValidity();
}
```

- 19. checkValidity() vs reportValidity()
  - **Concept Explanation (Sinhala):** මේ දෙක පටලවා නොගෙන මතක තියාගන්න මෙන්න මේ වගුව උදව් කරගන්න:

Method	What It Does	Use When
`checkValidity()`	නිහඬව check කරලා `true` හෝ `false` ලබාදේ.	අපේ code එක ඇතුළේ තීරණයක් ගන්න ඕන වූ විට.
`reportValidity()`	Check කරලා වැරැද්ද user ට popup එකක් විදිහට පෙන්වයි.	වැරැද්ද user ට පෙන්වන්න අවශ්‍ය වූ විට.

- 20. setCustomValidity()
  - **Concept Explanation (Sinhala):** අපිටම කියලා custom error message එකක් හදාගන්න මේ method එක පාවිච්චි කරනවා. අපි මේකට text එකක් දුන්න ගමන් browser එක සලකන්නේ මේ input එක invalid එකක් විදිහටයි.

  - **Code Block:**

    ```
    email.setCustomValidity("Use a company email address.");
    ```

- 21. Clearing Custom Validity (වැදගත්ම කොටස!)
  - **Concept Explanation (Sinhala):** අපි custom error එකක් දැම්මට පස්සේ, user ඒ වැරැද්ද නිවැරදි කරත් input එක දිගටම invalid විදිහටම තියෙනවා අපි ඒ custom error එක අයින් කරනකන්. Custom error එකක් අයින් කරන්න (clear කරන්න) අපි හිස් string එකක් (`""`) pass කරන්න ඕනේ: `email.setCustomValidity("");` හැමවෙලේම අලුතින් check කරන්න කලින් පරණ error එක clear කරලා ඉන්න අමතක කරන්න එපා!
  - **Code Block:**

    ```
    // නිවැරදි ක්‍රමය:
    email.setCustomValidity(""); // 1. පරණ වැරැද්ද clear කරනවා.

    if (!email.value.endsWith("@company.com")) { // 2. අලුත් value එක check කරනවා.
      email.setCustomValidity("Use a company email address."); // 3. වැරදි නම් අලුත් error එක දානවා.
    }
    ```

- 22. Custom Validation Example
  - **Concept Explanation (Sinhala):** පහත code එකෙන් email එක validate කරන්න reusable function එකක් හදන ආකාරය පෙන්වනවා.
  - **Code Block:**

    ```
    function validateEmail() {
      email.setCustomValidity("");

      if (email.value !== "" && !email.value.endsWith("@company.com")) {
        email.setCustomValidity("Use your company email address.");
      }

      return email.checkValidity();
    }
    ```

  - **Code Breakdown (Sinhala):** 1. `email.setCustomValidity("")`: මුලින්ම පරණ custom errors සේරම clear කරනවා. 2. `email.value !== "" &&`

`!email.value.endsWith("@company.com")` : Email field එක හිස් නැතිනම් සහ ඒක `@company.com` වලින් ඉවර වෙන්නේ නැතිනම් පමණක්: 3. `email.setCustomValidity(...)` : අපේ custom error message එක සෙට් කරනවා. 4. `return email.checkValidity()` : අවසානයේදී email එක valid ද නැද්ද යන්න `true` / `false` විදිහට return කරනවා.

Part F: The input Event

- ### 23. What Is the input Event?
  - **Concept Explanation (Sinhala):** User input field එකක් ඇතුළේ අකුරක් type කරන කරන වාරයක් පාසා හෝ value එකක් වෙනස් කරන හැම මොහොතකම ක්‍රියාත්මක වෙන event එකක් තමයි `input` කියන්නේ.
  - **Code Block:**

    ```
    email.addEventListener("input", () => {
      console.log(email.value);
    });
    ```

- ### 24. When to Use input
  - **Concept Explanation (Sinhala):** User ට live (එසැණින්) feedback ලබා දෙන්න ඕන හැම වෙලාවකම `input` event එක පාවිච්චි කරන්න පුළුවන්. උදාහරණ:
    - Live validation (අකුරු type කරද්දීම වැරදි පෙන්වීම).
    - Character counters (අකුරු කීයක් type කළාද කියා පෙන්වීම).
    - Live search/filters.

- ### 25. Live Custom Validation Example
  - **Concept Explanation (Sinhala):** User අකුරක් type කරන කරන සැරේට email එක check කරලා එසැණින් error එක clear කරන්න හෝ දාන්න මෙන්න මෙහෙම කරන්න පුළුවන්.
  - **Code Block:**

    ```
    email.addEventListener("input", () => {
      email.setCustomValidity("");
      if (!email.value.endsWith("@company.com")) {
        email.setCustomValidity("Use a company email address.");
      }
    });
    ```

Part G: The submit Event

- 26. What Is the submit Event?
  - **Concept Explanation (Sinhala):** Form එකක් submit වෙන්න උත්සාහ කරනකොට ක්‍රියාත්මක වෙන event එකක් තමයි `submit` කියන්නේ.
  - **Code Block:**

```
form.addEventListener("submit", event => {
  console.log("Form submitted");
});
```

- 27. How a Form Can Submit
  - **Concept Explanation (Sinhala):** Form එකක් ක්‍රම කිහිපයකින් submit වෙන්න පුළුවන්:
    1. Submit button එක click කිරීමෙන්.
    2. Input field එකක් ඇතුළේ ඉඳන් කීබෝඩ් එකේ `Enter` key එක එබීමෙන්.
    3. JavaScript මගින් submit trigger කිරීමෙන්.

    මේ නිසා කවදාවත් submit button එකේ `click` event එක විතරක් check කරන්න එපා. හැමවෙලේම form එකේ `submit` event එකට listen කරන්න.
  - **Code Block:**

```
// දුර්වල ක්‍රමය (Weak approach):
button.addEventListener("click", () => { ... });

// නිවැරදි ක්‍රමය (Better approach):
form.addEventListener("submit", event => { ... });
```

Part H: preventDefault()

- 28. What Is Default Browser Behavior?
  - **Concept Explanation (Sinhala):** HTML form එකක් submit වෙද්දී browser එකේ සාමාන්‍ය ස්වභාවය තමයි පිටුව refresh (reload) කිරීම හෝ වෙනත් page එකකට data යවා navigate වීම.

- 29. What Does preventDefault() Do?
  - **Concept Explanation (Sinhala):** `event.preventDefault()` මගින් browser එකේ සිදුවන ඒ සාමාන්‍ය ක්‍රියාවලිය (පිටුව reload වීම) තාවකාලිකව නවත්වනවා. එවිට අපිට JavaScript මගින් form එක validate කරලා අපිට ඕන විදිහට වැඩේ කරන්න ඉඩ ලැබෙනවා.
  - **Code Block:**

```
form.addEventListener("submit", event => {
  event.preventDefault();
});
```

- ### 30. Why preventDefault() Matters in Sprint 15
  - **Concept Explanation (Sinhala):** අපි JavaScript වලින් form එකක් validate කරනකොට පිටුව reload වුණොත්, අපේ console messages සේරම මැකිලා යනවා වගේම අපිට error messages පෙන්වන්නත් වෙලාවක් ලැබෙන්නේ නැහැ. ඒ නිසා `event.preventDefault()` අනිවාර්යයෙන්ම පාවිච්චි කරන්න ඕනේ.

Part I: Full Beginner Example (31 - 34)

- **Concept Explanation (Sinhala):** අපි දැන් සම්පූර්ණ උදාහරණයක් බලමු. අපේ rules මෙන්න මේවායි:
  - Email එක: required, email format එකට තියෙන්න ඕනේ, වගේම `@company.com` වලින් ඉවර වෙන්න ඕනේ.
  - Feedback එක: required, අඩුම තරමේ characters 10ක්වත් ඕනේ.
  - Submit වෙද්දී: පිටුව reload වෙන්න දෙන්න බෑ, errors පෙන්වන්න ඕනේ, valid නම් success message එකක් `aria-live` region එකක පෙන්වලා form එක reset කරන්න ඕනේ.
- **Code Block: HTML:**

```
<form id="feedbackForm">
  <label for="email">Company email</label>
  <input id="email" type="email" required
placeholder="name@company.com">

  <label for="feedback">Feedback</label>
  <textarea id="feedback" required minlength="10" placeholder="Write
at least 10 characters"></textarea>

  <button type="submit">Send</button>
  <p id="status" aria-live="polite"></p>
</form>
```

JavaScript:

```
const form = document.getElementById("feedbackForm");
const email = document.getElementById("email");
const feedback = document.getElementById("feedback");
const status = document.getElementById("status");

function validateEmail() {
  email.setCustomValidity("");
  if (email.value !== "" && !email.value.endsWith("@company.com")) {
```

```
    email.setCustomValidity("Use your company email address.");
  }
  return email.checkValidity();
}

email.addEventListener("input", () => {
  validateEmail();
});

form.addEventListener("submit", event => {
  event.preventDefault();

  const emailIsValid = validateEmail();
  const feedbackIsValid = feedback.checkValidity();

  if (!emailIsValid) {
    email.reportValidity();
    return;
  }

  if (!feedbackIsValid) {
    feedback.reportValidity();
    return;
  }

  status.textContent = "Form is valid. Feedback is ready to send.";
  form.reset();
});
```

Part J: JavaScript Example Explanation (35 - 51)

- **Concept Explanation (Sinhala):** ඉහත සකස් කළ JavaScript code එකේ හැම පේළියකම සිදුවන දේ තේරුම් ගනිමු:
  - **Selecting elements (35 - 38):** `document.getElementById` මගින් form, email input, feedback textarea, සහ status paragraph එක JavaScript variables වලට select කරගන්නවා.
  - **Creating reusable function (39):** `validateEmail()` ලෙස function එකක් හැදුවේ අපිට මේ check එක user type කරනකොට (`input` event එකේදී) වගේම form එක submit කරනකොටත් කියන තැන් දෙකේදීම පාවිච්චි කරන්න වෙන නිසා.
  - **Clearing old errors (40):** `email.setCustomValidity("")` මගින් පරණ custom errors clear කරනවා.
  - **Checking company rule (41):** `email.value !== ""` (හිස් නැත්නම්) සහ `!email.value.endsWith("@company.com")` (company email එකක් නොවේ නම්) යන තත්ත්වය check කර custom error එකක් දමනවා.
  - **Returning validity result (43):** `email.checkValidity()` මගින් email එකේ මුළු validity තත්ත්වය (`true` / `false`) ලබාදෙනවා.
  - **Live validation (44):** User type කරන හැම සැරේටම `validateEmail()` run වෙනවා.

- **On Submit (45 - 46):** `submit` event එක අල්ලාගෙන, `event.preventDefault()` මගින් page එක reload වෙන එක නවත්වනවා.
- **Checking both fields (47):** `emailIsValid` සහ `feedbackIsValid` variables දෙකට අදාළ validity values ගන්නවා. (`feedback.checkValidity()` එකෙන් check කරන්නේ HTML වල තියෙන `required` සහ `minlength="10"` නීති දෙකයි).
- **Handling invalid email (48):** Email එක invalid නම්, `email.reportValidity()` මගින් user ට error එක පෙන්වලා, `return` මගින් function එක එතැනින්ම නවත්වනවා (success message එක run වෙන්න දෙන්නේ නෑ).
- **Handling invalid feedback (49):** Feedback එක invalid නම්, ඒ වැරැද්ද පෙන්වලා, `return` කරනවා.
- **Showing success message (50):** හැමදේම valid නම්, `status.textContent` මගින් success message එක දානවා. (වැදගත්: user input පෙන්වන්න `innerHTML` පාවිච්චි කරන්න එපා, ආරක්ෂාව සඳහා `textContent` ම පාවිච්චි කරන්න).
- **Resetting form (51):** `form.reset()` මගින් form එකේ තියෙන සියලුම fields ආපහු හිස් කරනවා.

Part K: Form Submission Settings

- 52. action
  - **Concept Explanation (Sinhala):** `action` attribute එකෙන් කරන්නේ form එක submit වෙද්දී එහි ඇති data යැවිය යුත්තේ කුමන URL එකකටද කියා browser එකට පැවසීමයි. `action` එකක් නැතිනම් සාමාන්‍යයෙන් data යවන්නේ දැනට තිබෙන පිටුවේ URL එකටමයි.
  - **Code Block:**

    ```
    <form action="/feedback">
    ```

- 53. method
  - **Concept Explanation (Sinhala):** `method` attribute එකෙන් කරන්නේ form data එක යවන ආකාරය control කරන එකයි. ප්‍රධාන ක්‍රම දෙකක් තිබේ: `GET` සහ `POST`.
- 54. GET
  - **Concept Explanation (Sinhala):** `GET` ක්‍රමයේදී form data සියල්ල URL එක අගට එකතු කරලා යවනු ලැබේ (උදා: `/search?q=javascript`).
    - පොදුවේ පාවිච්චි කරන්නේ: search කරන්න, filter කරන්න හෝ data කියවන්න (reading data).
- 55. POST

  - **Concept Explanation (Sinhala):** `POST` ක්‍රමයේදී form data සියල්ල සඟවා request body එක ඇතුළේ යවනු ලැබේ.
    - පොදුවේ පාවිච්චි කරන්නේ: account එකක් හදන්න (signup), feedback යවන්න, හෝ data එකක් create/update කරන්න.

- 56. GET vs POST
  - **Concept Explanation (Sinhala):** සරලව මෙසේ මතක තබා ගන්න:

Method	Simple Meaning	Common Use
GET	Data ඉල්ලා සිටීම (Ask for data)	Search, filters, reading data
POST	අලුතින් data එකක් යැවීම (Send data to create/update)	Signup, feedback, checkout

- 57. enctype
  - **Concept Explanation (Sinhala):** `enctype` එකෙන් කියන්නේ data යවන්න කලින් ඒවා encode කරන්නේ කොහොමද කියන එකයි.
    - සාමාන්‍යයෙන් file uploads කරනවා නම් විතරක් අපි `enctype="multipart/form-data"` සෙට් කරන්න ඕනේ. සාමාන්‍ය forms සඳහා මේක අතින් දාන්න අවශ්‍ය නැහැ.

Part L: Accessibility in Sprint 15

- 58. Why Form Accessibility Matters
  - **Concept Explanation (Sinhala):** Accessibility (හැමෝටම පහසුවෙන් භාවිතා කළ හැකි බව) ඉතාම වැදගත්. අපි screen එකේ පෙන්වන success/error messages, පෙනීමේ අපහසුතා ඇති අය පාවිච්චි කරන screen readers වැනි assistive technologies වලට ඇහෙන්න සලස්වන්න ඕනේ.

- 59. aria-live="polite"
  - **Concept Explanation (Sinhala):** `aria-live="polite"` attribute එක දාපු element එකක් ඇතුළේ තියෙන text එකක් JavaScript වලින් වෙනස් කරපු සැනින්, screen reader එක මගින් ඒ අලුත් message එක user ට කියවා පෙන්වනු ලබයි.
  - **Code Block:**

```
<p id="status" aria-live="polite"></p>
```

Part M: Common Sprint 15 Mistakes

- 60. Mistake 1: Only Listening for Button Clicks
    - **Concept Explanation (Sinhala):** බොහෝ අය කරන වැරැද්දක් තමයි submit button එකේ click event එක විතරක් check කරන එක. නමුත් user කෙනෙක් input field එකක් ඇතුළේ ඉඳන් `Enter` එබුවොත් form එක submit වෙනවා, හැබැයි click event එක run වෙන්නේ නැහැ. **සෑමවිටම form එකේ `submit` event එකට listen කරන්න.**
- 61. Mistake 2: Forgetting preventDefault()
    - **Concept Explanation (Sinhala):** `event.preventDefault()` ලියන්න අමතක වුණොත්, form එක submit වෙද්දී page එක reload වෙන නිසා අපිට JavaScript validation හරියාකාරව කරගන්න වෙන්නේ නැහැ.
- 62. Mistake 3: Not Clearing Custom Validity
    - **Concept Explanation (Sinhala):** `setCustomValidity("")` මගින් පරණ error එක clear නොකළොත්, user වැරැද්ද හදාගෙන නිවැරදි අගයක් ඇතුළත් කළත් input එක දිගටම invalid විදිහටම පවතිනවා.
- 63. Mistake 4: Making Regex Too Complex Too Early
    - **Concept Explanation (Sinhala):** මුලදීම අතිශය සංකීර්ණ regex ලිවීමට උත්සාහ නොකරන්න. HTML built-in attributes වලට මුල් තැන දී, ඉතිරි කොටස සරල JavaScript string checks වලින් කරන්න.
- 64. Mistake 5: Showing User Input with innerHTML
    - **Concept Explanation (Sinhala):** කිසිම විටක user ඇතුළත් කරන data, page එකේ පෙන්වන්න `innerHTML` පාවිච්චි කරන්න එපා (එවිට malicious scripts run විය හැක - XSS vulnerability). හැමවෙලේම `textContent` පාවිච්චි කරන්න.
    - **Code Block:**

      ```
      // වැරදි ක්‍රමය:
      status.innerHTML = feedback.value;

      // නිවැරදි ක්‍රමය:
      status.textContent = feedback.value;
      ```
- 65. Mistake 6: Forgetting return After Invalid Input
    - **Concept Explanation (Sinhala):** වැරැද්දක් අහු වුණාට පස්සේ function එක එතනින් නවත්වන්න `return` එකක් දාන්න අමතක කරන්න එපා. නැතහොත් වැරදි තිබියදීම success code එක run වෙන්න පුළුවන්.
- 66. Mistake 7: Validating Only Once
    - **Concept Explanation (Sinhala):** එක සැරයක් විතරක් check කරලා නිකන් ඉන්න එපා. User දත්ත වෙනස් කරද්දී (`input` event) වගේම අන්තිමට submit කරද්දීත් (`submit` event) validation එක සිදුවිය යුතුයි.

Part N: Core Sprint 15 Patterns (67 - 70)

- **Concept Explanation (Sinhala):** මේවා තමයි ඔයා අනිවාර්යයෙන්ම දැනගෙන ඉන්න ඕන ප්‍රධානම coding patterns 4:
  1. **Basic Submit Pattern:** සරලවම form එකක් submit වෙද්දී checkValidity කරලා, invalid නම් report කරලා, valid නම් ඉදිරියට යන සරලම ක්‍රමය.
  2. **Custom Validation Pattern:** Reusable function එකක් ඇතුළේ මුලින්ම error clear කරලා, ඊටපස්සේ check කරලා අලුත් error එක සෙට් කරන ක්‍රමය.
  3. **Live Validation Pattern:** User type කරන අතරතුරේදී `input` event එක හරහා live validate කරන රටාව.
  4. **Submit Validation Pattern with Multiple Fields:** Fields කිහිපයක් තියෙනකොට එකින් එක check කරමින්, වැරැද්දක් හමු වූ සැනින් එතනින්ම `return` වී නවතින රටාව.
- **Code Block: 1. Basic Submit Pattern:**

```
form.addEventListener("submit", event => {
  event.preventDefault();
  if (!input.checkValidity()) {
    input.reportValidity();
    return;
  }
  console.log("Form is valid");
});
```

2. Custom Validation Pattern:

```
function validateInput() {
  input.setCustomValidity("");
  if (input.value !== "expected value") {
    input.setCustomValidity("Custom error message.");
  }
  return input.checkValidity();
}
```

3. Live Validation Pattern:

```
input.addEventListener("input", () => {
  validateInput();
});
```

4. Submit Validation with Multiple Fields:

```
form.addEventListener("submit", event => {
```

```
    event.preventDefault();

    const firstIsValid = firstInput.checkValidity();
    const secondIsValid = secondInput.checkValidity();

    if (!firstIsValid) {
      firstInput.reportValidity();
      return;
    }

    if (!secondIsValid) {
      secondInput.reportValidity();
      return;
    }

    status.textContent = "Form is valid.";
  });
```

Part O: Key Terms (71 - 85)

- **Concept Explanation (Sinhala):** මෙම පාඩමේදී පාවිච්චි වන ප්‍රධාන වචන සහ ඒවායේ සරල තේරුම මෙන්න:
  - **Form (71):** User ගෙන් දත්ත එකතු කරගන්නා HTML ව්‍යුහය (`<form>`).
  - **Input (72):** User ට දත්ත ඇතුල් කිරීමට දෙන field එක (`<input>`).
  - **Validation (73):** දත්ත නීති වලට අනුකූලද කියා පරීක්ෂා කිරීම.
  - **Constraint Validation API (74):** Browser එක සතු validation මෙවලම් සමූහය (`checkValidity` , `reportValidity` , `setCustomValidity`).
  - **checkValidity() (75):** Valid ද කියා නිහඬව පරීක්ෂා කර `true` / `false` ලබාදීම.
  - **reportValidity() (76):** Valid ද කියා පරීක්ෂා කර, වැරදි ඇත්නම් user ට පෙන්වීම.
  - **setCustomValidity() (77):** අපේම custom error message එකක් සෙට් කිරීම.
  - **setCustomValidity("") (78):** Custom error message එක ඉවත් (clear) කිරීම.
  - **input event (79):** User අකුරක් වෙනස් කරන හැම මොහොතකම ක්‍රියාත්මක වීම.
  - **submit event (80):** Form එක submit කරන්න උත්සාහ කරන විට ක්‍රියාත්මක වීම.
  - **preventDefault() (81):** Browser එකේ normal submit (page reload) වීම වැළැක්වීම.
  - **form.reset() (82):** Form එකේ සියලුම fields නැවත හිස් කිරීම.
  - **action (83):** Form data යැවිය යුතු URL එක.
  - **method (84):** Data යවන ක්‍රමය (`GET` හෝ `POST`).
  - **enctype (85):** Data encode කරන ආකාරය (විශේෂයෙන් file upload වලදී).

Part P: Minimum Passing Standard for Sprint 15 (86 - 92)

- **Concept Explanation (Sinhala):** ඔයා ඊළඟ කොටසට යන්න කලින් මෙන්න මේ code එකේ තියෙන හැම පේළියකම තේරුම තනියම පැහැදිලි කරන්න පුළුවන්ද කියලා බලන්න. ඔයාට පුළුවන් නම් ඔයා Sprint 15 සාර්ථකව නිම කරලා තියෙනවා!
- **Code Block:**

```
form.addEventListener("submit", event => {
  event.preventDefault();

  if (!input.checkValidity()) {
    input.reportValidity();
    return;
  }

  console.log("Form is valid");
});
```

- **Code Breakdown (Sinhala):** - `form.addEventListener("submit", ...)` : Form එක submit වෙනකොට මේ function එක run කරන්න කියලා listen කරනවා.
  - `event.preventDefault()` : Browser එක page එක reload කරන එක නවත්වනවා.
  - `input.checkValidity()` : Input එක valid නම් `true` දෙනවා, වැරදි නම් `false` දෙනවා.
  - `input.reportValidity()` : වැරැද්දක් තියෙනවා නම් ඒක user ට පෙන්වනවා.
  - `return` : එතනින් එහාට code එක run වෙන එක නවත්වනවා.

Part Q & R: Practice Task & Summary (93 - 97)

- **Concept Explanation (Sinhala):** අවසාන වශයෙන්, මෙන්න මේ Practice Task එක තනියම කරලා බලන්න.

  Task Rule:

  - Email field එක: required, type="email", වගේම `@company.com` වලින් ඉවර වෙන්න ඕනේ.
  - Feedback field එක: required, අඩුම characters 10ක් ඕනේ.
  - Submit: reload වීම නවත්වන්න, errors පෙන්වන්න, success message එක `aria-live` region එකක පෙන්වන්න.

  Final Golden Rules to Remember (97):

  1. හැමවෙලේම නිවැරදි HTML form elements පාවිච්චි කරන්න.
  2. HTML built-in validation attributes මුලින්ම යොදන්න.
  3. Custom validation අවශ්‍ය නම් පමණක් JavaScript යොදාගන්න.
  4. Button click වෙනුවට form එකේ `submit` event එකට listen කරන්න.
  5. JavaScript වලින් control කරනකොට අනිවාර්යයෙන්ම `preventDefault()` දාන්න.

6. `setCustomValidity("")` මගින් පරණ errors clear කරන්න අමතක කරන්න එපා.
7. ආරක්ෂාව සඳහා හැමවෙලේම `textContent` පාවිච්චි කරන්න.
8. Screen readers සඳහා `aria-live` regions භාවිතා කරන්න.

- **Code Block: Complete Practice Solution:**

```
<form id="feedbackForm">
  <label for="email">Company email</label>
  <input id="email" type="email" required>

  <label for="feedback">Feedback</label>
  <textarea id="feedback" required minlength="10"></textarea>

  <button type="submit">Send</button>
  <p id="status" aria-live="polite"></p>
</form>

<script>
  const form = document.getElementById("feedbackForm");
  const email = document.getElementById("email");
  const feedback = document.getElementById("feedback");
  const status = document.getElementById("status");

  function validateEmail() {
    email.setCustomValidity("");
    if (email.value !== "" && !email.value.endsWith("@company.com")) {
      email.setCustomValidity("Use your company email address.");
    }
    return email.checkValidity();
  }

  email.addEventListener("input", () => {
    validateEmail();
  });

  form.addEventListener("submit", event => {
    event.preventDefault();

    const emailIsValid = validateEmail();
    const feedbackIsValid = feedback.checkValidity();

    if (!emailIsValid) {
      email.reportValidity();
      return;
    }

    if (!feedbackIsValid) {
      feedback.reportValidity();
      return;
    }

    status.textContent = "Form is valid. Feedback is ready to send.";
    form.reset();
```

```
  });
</script>
```

- **Code Breakdown (Sinhala):** - මුළු code එකේම සාරාංශය: HTML වලින් rules පනවනවා, JavaScript වලින් ඒවා check වෙන වෙලාව (submit/input) පාලනය කරමින්, user ට පැහැදිලිව වැරැද්ද හෝ සාර්ථකත්වය පෙන්වනවා!